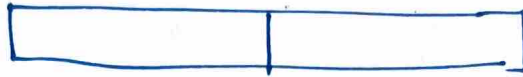


So we are looking at divide and conquer algorithm.

The basic idea of which is that, if I have an input say an array, but it could be absolutely anything. Divide it into two parts.



Solve for the left and right halves and put the solution back together. Summarized as follows

- Divide input
- Solve each part
- Put them together

to get solution for the larger problem. Often it pays to divide the array into equal parts. We will see an example why this is so.

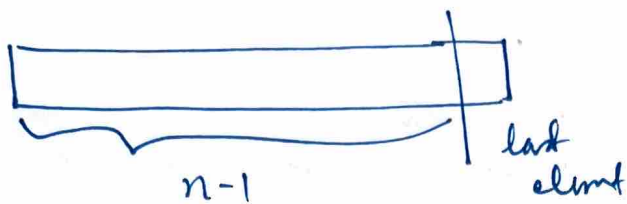
The next problem that we are going to consider is very familiar to all of us is ~~sort~~ sorting.

Only we are going to look at it from the new design principle that we have learnt and we are try and apply those to sorting.

Let us take a simplest case.

(2)

Solnig:



we have an array of n elements we would like to sort the elements in increasing order. Let us assume that array elements are distinct.

This is not going to

change any of our design principle or that of the algorithm that we ~~came~~ come up with but it is just to keep our mind less cluttered.

So you have an array of n elements and you would like to sort them in increasing order.

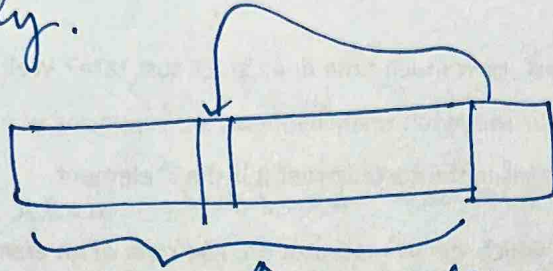
Let us put the first design principle into practice which is supposing you can solve it for smaller arrays, how do you ~~increase~~ extend it to a bigger array? Most natural smaller array let us say is the 1st $n-1$ elements. Supposing I could ~~the~~ solve the problem of sorting the array for the 1st $n-1$ elements, how do I extend it for the n element array. Refer to

The figure:

So you sort the first $n-1$ elements. ~~then you put the last element in its proper~~

(3)

may be by recursion say and you now want to place the last element in its right position. So I guess most of you know binary search and can be used to ^{identify} find the position very easily.



this position is valid and the position of the last ~~pos~~ element is identified. Now what you have to do to insert the new element in its right position, you have to move some elements to the right to make room for the new element.

So here is the algorithm.

- Recursively sort the 1st $n-1$ elements.
- Figure out where the last element goes in
- and then put it in the place

This is called insertion sort. You might have learnt insertion sort earlier. @ you know what insertion sort is all about, but here @ we are trying to

understand, how it is that people came up with these algorithms, so that when you face with a new problem, you can come up with your own algorithm.

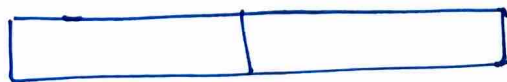
So how much time does this take well, if we just look at the number of comparisons, to put the last element in its place the number of comparison was ' $\log n$ '. since we have used binary search and every other elements took less no of comparison, for the i th element in the array took ' $\log i$ ' no of comparisons. so the total time is in fact $\approx n \log n$ roughly. But the main step here, which takes time is not comparisons but this movement. let us look at the last step, once I find the position of the last element all the elements from that position had to move one place to the right. Now the place I find could be the 1st position of the array in which case we need to move the entire array by one position.

So, while we take just ' $\log n$ ' line to find the right position, to move the elements we may take n units of time and this is the expensive steps in the insertion sort. and if implemented in this manner, each line you take $O(n)$ steps. For the i th steps you $O(i)$ time to move it to the right in the worst case and the running time of the algorithm would $O(n^2)$.

I leave it for you to see if you can me better data structure. So that you can sort of avoid the movement, because the number of comparisons here is only $n \log n$.

Let us put our other design paradigms into place. and the ~~the~~ paradigm that we are going discuss now is divide and conquer.

Let us try another technique to sort an array of n elements.



we divide it in two equal halves, sort each piece and then put them together.

(6)

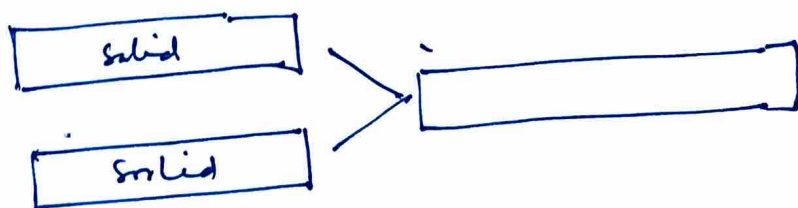
So the algorithm is clear except the step of putting them together.

- During the array into two pieces ~~→ no sort~~ needs no repeat

- Sorting each $\rightarrow$ recursion takes care of it

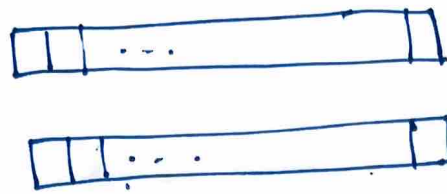
The crucial step here is ~~two~~ to put the two sorted arrays together. Let us focus on ~~them~~ this step. Because ~~as~~ once we implement this the algorithm is ready and you can program it.

This step entails the following we have two sorted arrays.

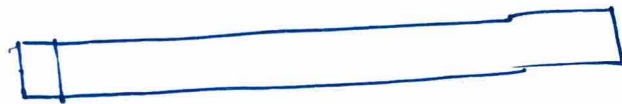


and we need to combine them ~~is~~ to get one big sorted array. So this is the problem that we would like to solve. and this is called merging two sorted arrays or test.

So we are given elements in ascending order and they are stored in two arrays.

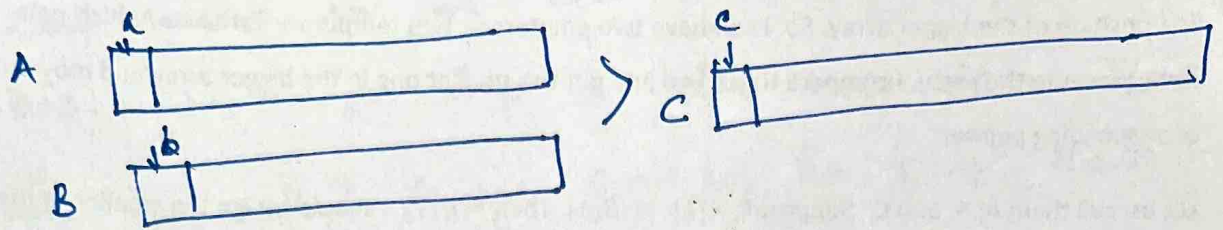


and we would like to build a bigger array that contains element from both the arrays and also in ascending order. The procedure is fairly natural



This array is the output. Now which ~~old~~ element will occupy the 1st location in the bigger array. it should be the smallest among the 1st two elements of the smaller arrays. So what I do is I compare the 1st elements of these two arrays or subarrays and put the smaller of them ~~into~~ at the 1st position in the bigger array. To do this we can take help of two pointers or two temporary variables that point to some places in the array. We compare the elements pointed by the two pointers and put the smaller one in the next position of the bigger array. and move the pointers accordingly

Let us name the arrays ~~a~~ and pointer as follows.



Supposing $A[i] < B[i]$ then $C[i] = A[i]$ and move pointer in A and C.

In general let us call the pointer in the array as a, the pointer in the array B as b and the pointers in the Array C as c. So the generic

step would be

if $A[a] < B[b]$ then $C[c] = A[a]$..

and then $a++$ and $c++$..

So this is the algorithm. I just essentially somehow scan the these two arrays and keep filling in the elements in the Big array C.

So this is merging and you can write code in any programming language of your choice if needed you can do this if you have two sorted list instead of arrays. The idea is very simple

You just need to have two pointers point to two sorted list, you compare the values

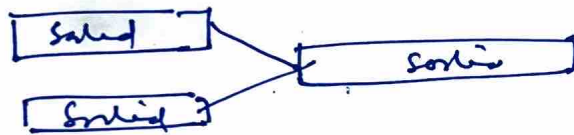
partitioned by these two pointers and put the smaller value in the new list and update the pointers. So merge step is also now done. This algorithm is called Merge sort by the way. Now

the question is how much time does this take?

The algorithm is



- Divide into two pieces
- Sort each
- put them together



The divide step will not take any time

Sorting each is a recursive step. We only need to figure out how much time does it take to put the two sorted arrays into one.

Suppose $T(n)$ is the time for an array of size n . By the way we will again focus on the number of comparisons made, because even here, every other operation like moving the pointer etc is bounded by the number of comparisons.

the total time is constant times the number of comparisons. So we can just focus on the number of comparisons only. & let $T(n)$ be the number of comparisons made we would like to bound the total no of comparisons for needed for an array of size n .

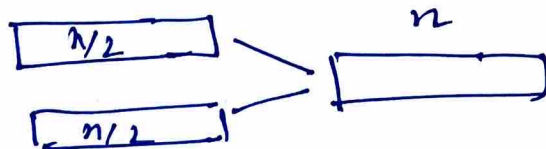
well divide step does not take any time. There are two subproblem of equal size ~~to~~, and there is merge step

$$T(n) = 2T(n/2) + \text{time to merge}$$

{ we will not worry about floors and ceilings for ease of calculation

Now how much time does merge step take on an array of size n . This is something we need to figure out now.

Merge



we have two arrays of size $n/2$

and we merge them to get an array of size n

How much time this procedure will take?

There are many ways of figuring this out.

Let us discuss one way, the first way of doing it. This is most way of doing it. Then we will discuss another way of doing it using design principle that we already seen and analyse it that way also.

1st way

So what is the each step? In each step we compare two elements and one element gets filled in the array C.

The crucial sort of observation here is that each time you make a comparison, one element gets filled in the array C.

How many elements gets filled in the big array C? The answer is n , the total no of elements

So, the total no of comparison we make is almost n .

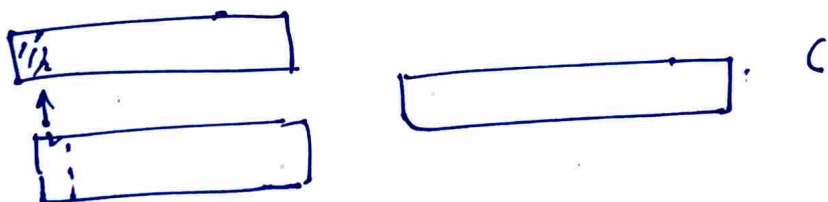
So we can conclude that the total no of comparison made in the merging step is bounded by n . and the reason is for each comparison, we add an element in the array C.

Now, we can rewrite the recurrence for the merge sort algorithm as follows

$$T(n) = 2 \cdot T(n/2) + n$$

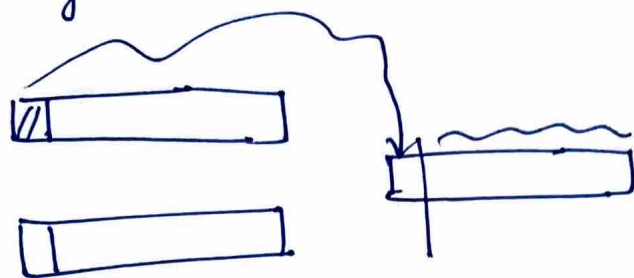
We can solve this recurrence and get the running time for merge sort, that is fine but let us do again do the merging differently

So, we are now going to 'apply' our design principle which is 'induction'. If we can merge two smaller arrays how do we merge larger arrays. So here are our two arrays



I know the 1st element in C is the smaller of the two elements. Let's say the shaded one is smaller so it will be put in C and the input now became 1 less elements. The size of the two arrays may not be same. Supposing I solve the problem for smaller input

can I solve the problem for larger input. well the answer is yes.



Shaded is the smaller of the two and now we have two arrays here with size smaller than the earlier. one element is removed and put in the bigger array C. Remaining portion of the subarray are recursively considered to fall in the remaining portion of the array C. so this is the other way of doing this using recursion.

How much time does this way of doing the merging take? lets write the recurrence here

$$T(m_1, m_2) =$$

m_1 is the size of the 1st array and m_2 is the size of the 2nd subarray

{ the array may have of different size

after the 1st comparison the size of the 1st array of the subarray decreases by one

So let us rather write the recurrence on the ~~sum~~ sum of the elements of both the subarrays

So

$$T(n) = T(n-1) + 1$$

$$T(2) = 1.$$

$$T(n) = n-1$$

where $n = m_1 + m_2$

where does this 1 come from? this comes from the 1st comparison.

So to merge two sub-arrays, whose the sum of two sizes is n , the number of comparisons needed is $n-1$.

So, there are the two ways of doing the merge steps.

Now ~~let~~ let us go back to the merge sort ~~algor~~ recurrence and see how much time does the merge sort algorithm takes?

$$T(n) = 2T(n/2) + O(n)$$

Let us just put n instead of $O(n)$ you can also put a constant time n and calculate this. It will make no difference

These constant just came out of the calculation

So we have the recurrence as

$$T(n) \leq 2 T(n/2) + n$$

$$T(2) = 1$$

$$T(n) = 2 T(n/2) + n$$

$$= 2 \left\{ 2 T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right\} + n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + n + n$$

$$\vdots \quad \vdots \quad \vdots$$

if we do it for i times.

$$= 2^i T\left(\frac{n}{2^i}\right) + i n$$

$$\Rightarrow$$

$$= \frac{n}{2} + i n$$

$$= \frac{n}{2} + n \log_2(n/2)$$

$$\approx O(n \log n)$$

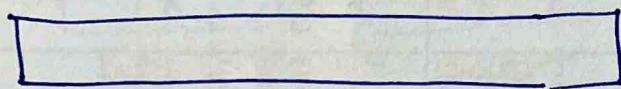
$$\frac{n}{2^i} = 2$$

$$2^{i+1} = n$$

$$i = \log_2(n/2)$$

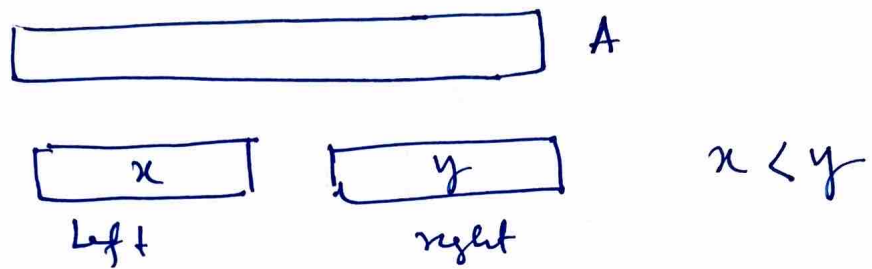
So the time taken by merge sort is $n \log n$

There is one more way of applying divide and conquer strategy on arrays to sort, and it in fact gives rise to another well known algorithm, which you may have studied. Let's see this.



- Here is the array and I still want to apply divide and conquer strategy here. So I want to divide it into two arrays, somehow work with each part and put the solution back together.
- But the way I do will be slightly different in this case. While dividing itself I start rearranging the array elements. So the array will be divided as in the previous case. In the previous case we just picked up the array and just divided arbitrarily into two parts of roughly equal size.

This time we are going to be more careful.
what we ~~want~~^{want this} line is every element landing up
in the left is ~~smaller~~ than the every element
landing up in the right



If x is in the left half and y is in the right half
we know that $x < y$. It is also to be noted that
all the elements in the array must occur in one of
the subarrays.

So somehow we have divided the
array into two parts left and right with the condition
that elements in the left are smaller than the
elements in the right.

Now once we somehow divide
the array into two ~~parts~~ halves ~~in the~~ we just
recurse on these two parts. We sort left
separately we sort right separately and see

what happens once we do this. Once we do this I claim that the entire array is now sorted. why is that so?

well, the left portion is sorted, the right portion is ~~also~~ sorted, and I also know that every element on the left is smaller than the every element ~~of~~ on the right. The largest element in the left half is smaller element on the ~~right~~ smallest ~~on~~ element in the right. So when I look ~~at~~ the entire array you can check that the entire ~~is~~ array is now sorted.

This is also an example of divide and conquer strategy as ~~as~~ in the case of Merge Sort but the difference is the divided step where we did most of the work.

The divide step (which ~~we~~ I still need to specify how) somehow divided the array into two parts left and right so that elements in the left are smaller than the elements in the right.

Now if it is recursion you just recurse on these 2 parts and recursion does it for you, the third step which is putting together is trivial, you do not have to do any work here. Both of them are solved when you put them together, ~~in fact~~ the entire array will be obviously sorted.

The only things we need to figure out is how to divide the input so that everything on the left is smaller than everything on the right. I guess many of you have noticed this algorithm that you have seen this algorithm before and it is called quick sort. But let me emphasize that earlier you were told what these algorithms are. you were maybe given the codes for this algorithm.

Our emphasis here is to see how do people come up with these algorithms. how does somebody come up with an algorithm ^{called} quick sort. and this is how they come up with the algorithm.

So the idea is, you start with divide and conquer at the back of your mind and you figure out how to put the paradigm into place

Now the question is how do you split an array left and right ~~that~~ is such that all the elements in the left ~~subarray~~ is smaller than all the elements of the right half that is to be answered.

well, you pick a ~~pivot~~ some element in the array which you call as pivot element. Now you need to arrange all the elements in the array such that all the elements in the left are smaller than the pivot and all the element on the right are greater ~~that~~ than or equal to the pivot. The smallest element in the right is greater than the pivot.

So let me just write this down so that we have the quick sort in front of us.

Quick sort

- Pick a pivot p
- Divide into two parts

elements ~~small~~ smaller than p and elements greater than p . (these are the two parts)

- Recurse and put them together (last step)

The difference between the merge sort and the quick sort is essentially the following

→ In quick sort divide step is where you spend most of your time. Putting them together is easy

→ In merge sort dividing into two parts were easy. Putting them together is where you took time.

Now let us see how much time does quick sort take?

well, it depends on which element you pick as pivot? Let us see what really happens. Suppose you pick the i th element

as the pivot. Now what do I mean by i th element? I mean the i th element in the sorted order. Let us make a definition here.

rank of an element: It is the position of the element in the sorted order. Remember we are assuming that each element in the array is distinct and so each element has a distinct rank.

If there are i element smaller than an element ' x ' say, then rank of x is $i+1$. The minimum element has the rank 1 and the maximum element has the rank n , which is the size of the array. So the rank of an element ~~is~~ is its position in the array.

As I said earlier that the time taken by quick sort depends on the rank of the pivot. So let us see. Suppose the rank of the pivot is i . Then there will be $(i-1)$ elements in one side and $(n-i)$ element on the other side. So the time taken by quick sort can be

written as

$$T(n) = T(i-1) + T(n-i) + \text{the time taken to partition the array}$$

Now what would be the time taken to divide the array in such a manner?

How do we partition the array? well we compared each element with the pivot and put the lesser ones on the left and the larger ones on the right. So the no of comparisons we made is $(n-1)$.

So we can rewrite the recurrence as follows.

$$T(n) \leq T(i-1) + T(n-i) + n$$

For ease of calculation we say it $\leq n$. It will not bother us much.

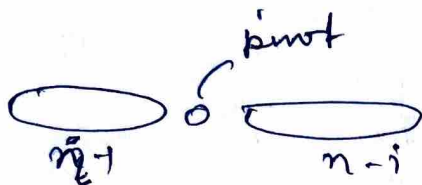
So what is the solution to this recurrence? clearly it depends upon i , and for various values of i it varies.

For instance if we pick the minimum element as pivot.

then the recurrence would be.

$$T(n) \leq T(n-1) + n$$

There would be nothing on the left and $n-1$ on the right



~~if we~~ and if we keep picking the minimum element then

$$T(n) = T(n-1) + n$$

$$= T(n-2) + (n-1) + n$$

$\vdots$

$$= 1 + 2 + \dots + n = \frac{n(n+1)}{2} = O(n^2)$$

So if the pivot turns out to be the min element in each step the total time taken by the quick sort algorithm is $O(n^2)$

Now if we keep increasing the rank lets say pivot is the 2nd element, the time decreases. It will keep on decreasing till you reach the middle element.

let us see what happens if

we pick roughly the middle element as the pivot. Then we will have roughly two subproblems of equal size.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

This solution to the recurrence is familiar to you and the solution is

$$T(n) = O(n \log n)$$

Let us put the cases side by side

$$T(n) = T(i-1) + T(n-i) + n$$

$$i=1 : O(n^2)$$

$$i \approx n/2 : O(n \log n)$$

Now if we ~~plot~~ ^{plot} $T(n)$ versus i , it initially falls and it falls still we reach $n/2$ and then it starts rising. If we pick the maximum element, the behavior is exactly similar to picking the minimum. The array splits much the same way. So the worst case time is $O(n^2)$ and if you manage to pick the pivot

to be the middle element at each stage,
then the time taken by the quick sort is
 $n \log n$. But this ~~implementation~~ ^{implementation} of quick sort, the
running time is $O(n^2)$ as we don't know ~~the~~ ^{the} ~~rank~~ ^{rank}
what would be the rank of the pivot elem.

But if somehow we could pick
an ~~element~~ ^{element} of rank $n/2$ efficiently, ~~what by mean~~
then running time of quick sort would ~~be~~ ^{be} ~~be~~ ^{be}
 $O(n \log n)$. But what do we mean by
efficiently? this means the recursion should
not change much. I know I get $n \log n$
by the ~~for~~ following recurrence

$$T(n) = 2T(n/2) + n \quad \text{in fact we}$$

could write $T(n) = 2T(n/2) + C \cdot n$ $\left\{ \begin{array}{l} \text{for some} \\ \text{constant} \\ C \end{array} \right.$

& still we get $O(n \log n)$

Now if we can somehow find middle
element of an array, which means an element
of rank roughly $n/2$ then I can implement

quick sort to take $O(n \log n)$ time.

So there is can be an implementation of quick sort which takes $O(n \log n)$ time, provided we can find the element of rank $n/2$ in linear time.

Hopefully there is an algorithm to find the element of rank roughly $n/2$ in linear time. and this is what we are going to discuss next. Before we actually do that, let me point out the fact that in the recurrence, it was desirable that the two parts were roughly equal size. If they were not ^{of} equal size then the time taken was proportionally higher. So in the divide and conquer strategy we often try to see if we can somehow split the input into exactly two parts. The way this recurrence works the time taken is usually smaller if ~~we have~~ the input is divided into two equal halves.

bestcase

(t)

So our job is to figure out how to pick the element of rank $n/2$ in linear time.

~~So let me state this problem~~

Definition: The element rank $n/2$ is called median. we also could take $\lfloor n/2 \rfloor$ or $\lceil n/2 \rceil$ as median - it does not matter. So our problem is given an array of n elements and we want to find the median is the element of rank roughly $n/2$

Well, the easy way to do is to sort the array and pick the middle element that takes time $O(n \log n)$ (if we can use merge sort)

But that is not good enough. Can we do better? we need something that does the thing in linear time.

And the answer to this question is yes.

You can in fact find the median in linear time. and this will be our first non-trivial solution algorithm. The algorithm is really smart and I hope you appreciate the beauty of the algorithm.